Universidad Nacional de Colombia

Programación Orientada a Objetos 2026-2S

Actividad 4: Excepciones y Archivos
Valor 20 %

Estudiante: Alejandro Ramos Rincón

Docente: Walter Hugo Arboleda Mazo

URL del Repositorio en GitHub:

https://github.com/alramosr/Actividad4_POO

Noviembre 2026

Índice

Consideraciones generales **2**

1. Ejercicio 6.4: Excepciones **3**
1.1. Código fuente 3
1.2. Diagrama de clases 6
1.3. Diagrama de casos de uso 7
1.4. Interfaz de usuario 7

2. Ejercicio 6.5: Lanzamiento de excepciones **8**
2.1. Código fuente 8
2.2. Diagrama de clases 12
2.3. Diagrama de casos de uso 12
2.4. Interfaz de usuario 13

3. Ejercicio 6.6: Catchs múltiples **14**
3.1. Código fuente 14
3.2. Diagrama de clases 18
3.3. Diagrama de casos de uso 18
3.4. Interfaz de usuario 18

4. Ejercicio 6.7: Validación de campos **20**
4.1. Código fuente 20
4.2. Diagrama de clases 28
4.3. Diagrama de casos de uso 28
4.4. Interfaz de usuario 29

5. Ejercicio 6.8: Lectura de archivos **31**
5.1. Código fuente 31
5.2. Diagrama de clases 35
5.3. Diagrama de casos de uso 35
5.4. Interfaz de usuario 36

Consideraciones generales

Los ejercicios 6.4 a 6.8 del libro *Ejercicios de programación orientada a objetos con Java y UML* de Leonardo Bermón Angarita fueron desarrollados en Java mediante interfaz gráfica de usuario (librería *swing*), como lo pide la actividad. Cada ejercicio está organizado en su propio paquete (`Ejercicio6_4` a `Ejercicio6_8`) y cuenta con una clase `Main` que contiene el método `main`.

Los datos que en el libro se leen por teclado con `Scanner` se ingresan en campos de texto, y lo que el libro imprime en la consola se muestra en la ventana. Por eso, cuando el libro captura `InputMismatchException` (dato no numérico leído con `Scanner`), aquí se captura su equivalente al convertir el texto de un campo: `NumberFormatException`. Las excepciones se muestran en cuadros de diálogo o en la ventana, con su tipo y su mensaje.

Las clases del modelo implementan métodos *get* y *set* para cada uno de sus atributos; las ventanas extienden `JFrame`, implementan `ActionListener` y tienen como atributos sus componentes gráficos. Para facilitar la lectura, **en este documento se omiten los métodos *get* y *set***, tanto en el código como en los diagramas de clases; el código completo se encuentra en el repositorio de GitHub.

Para cada ejercicio se presentan el código fuente, el diagrama de clases, el diagrama de casos de uso y las imágenes de la interfaz de usuario. Los diagramas se elaboraron con draw.io y las imágenes de la interfaz son capturas de los programas en ejecución.

1. Ejercicio 6.4: Excepciones

El libro pide determinar qué imprime el método `main` de la clase `PruebaExcepciones`, que tiene dos bloques *try*: el primero lanza una `ArithmeticException` (división por cero) y el segundo una excepción general al llamar un método sobre un objeto nulo; cada bloque tiene sus *catch* y su *finally*. En la interfaz gráfica el programa se ejecuta con un botón y lo que imprime se muestra en la ventana, en el mismo orden en que se ejecutan los bloques.

1.1. Código fuente

Código 1: PruebaExcepciones.java

```
package Ejercicio6_4;

//Creación de la clase PruebaExcepciones (lanza diferentes excepciones en situaciones específicas del programa).
public class PruebaExcepciones {

    //Creamos un método con los dos bloques try del libro que generan excepciones.
    public static String EjecutarPrueba(){

        //Inicializamos una variable para guardar los mensajes que se van generando.
        String Resultado = "";

        //Primer bloque try.
        try {
            Resultado = Resultado + "Ingresando al primer try\n";
            double Cociente = 10000 / 0; //Se lanza una excepción.
            Resultado = Resultado + "Después de la división\n"; //Esta instrucción nunca será ejecutada.
        } catch (ArithmeticException Error){
            //Se captura la excepción y se guarda este mensaje.
            Resultado = Resultado + "División por cero\n";
        } finally {
            //La sentencia finally siempre se ejecuta, ocurra o no una excepción.
            Resultado = Resultado + "Ingresando al primer finally\n";
        }

        //Segundo bloque try.
        try {
            Resultado = Resultado + "Ingresando al segundo try\n";
            Object Objeto = null;
            Objeto.toString(); //Se lanza una excepción.
            Resultado = Resultado + "Imprimiendo objeto\n"; //Esta instrucción nunca será ejecutada.
        } catch (ArithmeticException Error){
            //La excepción lanzada no es de este tipo.
            Resultado = Resultado + "División por cero\n";
        } catch (Exception Error){
            //Se captura la excepción y se guarda este mensaje.
            Resultado = Resultado + "Ocurrió una excepción\n";
        } finally {
            //La sentencia finally siempre se ejecuta, ocurra o no una excepción.
```

```
39              Resultado = Resultado + "Ingresando al segundo finally\n";
40          }
41
42          //Retornamos los mensajes generados.
43          return Resultado;
44      }
45  }
```

Código 2: VentanaPrincipal.java

```
1   package Ejercicio6_4;
2
3   //Importamos las clases necesarias para la interfaz gráfica y los
        eventos.
4   import java.awt.Container;
5   import java.awt.Font;
6   import java.awt.event.ActionEvent;
7   import java.awt.event.ActionListener;
8   import javax.swing.JButton;
9   import javax.swing.JFrame;
10  import javax.swing.JLabel;
11  import javax.swing.JScrollPane;
12  import javax.swing.JTextArea;
13
14  //Creación de la clase VentanaPrincipal (ventana para ejecutar el
        programa PruebaExcepciones y ver su resultado).
15  public class VentanaPrincipal extends JFrame implements ActionListener {
16
17      //Sección para declaración de atributos.
18      Container Contenedor; //Contenedor de los elementos gráficos.
19      JLabel Seleccion, TituloResultado; //Etiquetas de la ventana.
20      JButton Prueba, Limpiar; //Botones para ejecutar el programa y para
            limpiar el resultado.
21      JTextArea Resultado; //Área de texto donde se muestra lo que imprime
            cada programa.
22      JScrollPane Desplazamiento; //Barras de desplazamiento del área de
            texto.
23
24      //Creamos el constructor de la ventana.
25      public VentanaPrincipal(){
26
27          //Creamos los componentes gráficos de la ventana.
28          Inicio();
29
30          //Establecemos el título, el tamaño y la posición de la ventana.
31          setTitle("Excepciones");
32          setSize(560, 400);
33          setLocationRelativeTo(null);
34
35          //Establecemos que el botón de cerrar termina la aplicación y
                que el tamaño no se puede cambiar.
36          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
37          setResizable(false);
38      }
39
40      //Creamos un método para crear la ventana con sus componentes grá
            ficos.
41      private void Inicio(){
```

```
42
43          //Obtenemos el panel de contenidos de la ventana y le quitamos
                el layout.
44          Contenedor = getContentPane();
45          Contenedor.setLayout(null);
46
47          //Establecemos la etiqueta que indica qué hacer.
48          Seleccion = new JLabel();
49          Seleccion.setText("Presione el botón para ejecutar el programa
                PruebaExcepciones:");
50          Seleccion.setBounds(20, 15, 400, 23);
51
52          //Establecemos el botón para ejecutar el programa y le agregamos
                el oyente de eventos.
53          Prueba = new JButton();
54          Prueba.setText("Ejecutar PruebaExcepciones");
55          Prueba.setBounds(20, 45, 230, 25);
56          Prueba.addActionListener(this);
57
58          //Establecemos el área de texto donde se muestra el resultado de
                la ejecución.
59          TituloResultado = new JLabel();
60          TituloResultado.setText("Resultado de la ejecución:");
61          TituloResultado.setBounds(20, 85, 300, 23);
62          Resultado = new JTextArea();
63          Resultado.setEditable(false);
64          Resultado.setFont(new Font("Monospaced", Font.PLAIN, 13));
65          Desplazamiento = new JScrollPane(Resultado);
66          Desplazamiento.setBounds(20, 110, 500, 200);
67
68          //Establecemos el botón Limpiar y le agregamos el oyente de
                eventos.
69          Limpiar = new JButton();
70          Limpiar.setText("Limpiar");
71          Limpiar.setBounds(420, 320, 100, 25);
72          Limpiar.addActionListener(this);
73
74          //Añadimos cada componente gráfico al contenedor de la ventana.
75          Contenedor.add(Seleccion);
76          Contenedor.add(Prueba);
77          Contenedor.add(TituloResultado);
78          Contenedor.add(Desplazamiento);
79          Contenedor.add(Limpiar);
80      }
81
82      //Creamos un método para gestionar los eventos generados en la
            ventana.
83      @Override
84      public void actionPerformed(ActionEvent Evento){
85
86          //Si se pulsa el botón Ejecutar PruebaExcepciones se ejecuta el
                programa y se muestra lo que imprime.
87          if (Evento.getSource() == Prueba){
88              Resultado.append("> PruebaExcepciones\n" + PruebaExcepciones
                    .EjecutarPrueba() + "\n");
89          }
90
91          //Si se pulsa el botón Limpiar se borra el resultado.
```

```
92          if (Evento.getSource() == Limpiar){
93              Resultado.setText("");
94          }
95      }
96  }
```

Código 3: Main.java

```
1   package Ejercicio6_4;
2
3   //Creación clase Main.
4   public class Main {
5       public static void main(String[] args){
6
7           //Creación de la ventana principal.
8           VentanaPrincipal MiVentanaPrincipal = new VentanaPrincipal();
9
10          //Hacemos visible la ventana principal.
11          MiVentanaPrincipal.setVisible(true);
12
13      }
14  }
```

1.2. Diagrama de clases

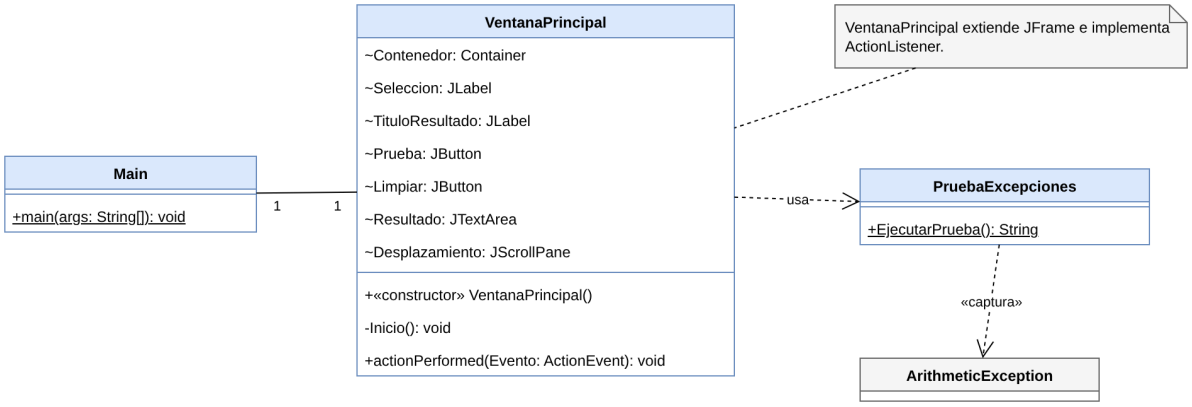


Figura 1: Diagrama de clases del ejercicio 6.4 (sin métodos get y set).

1.3. Diagrama de casos de uso

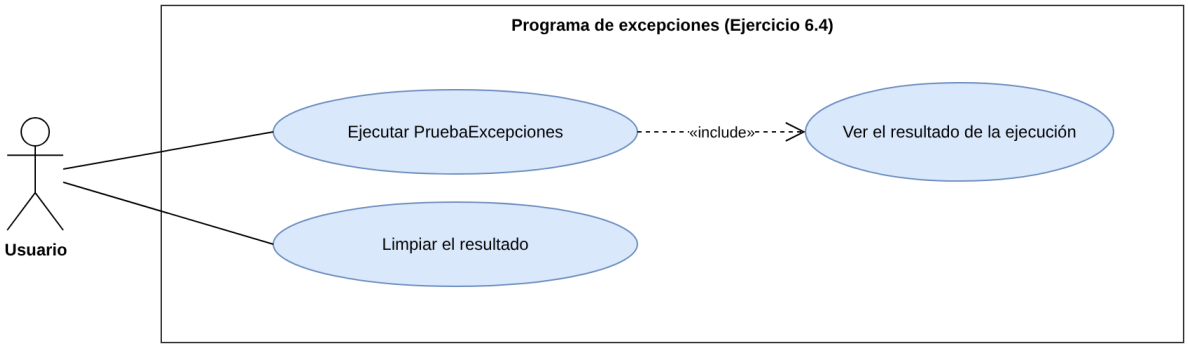


Figura 2: Diagrama de casos de uso del ejercicio 6.4.

1.4. Interfaz de usuario

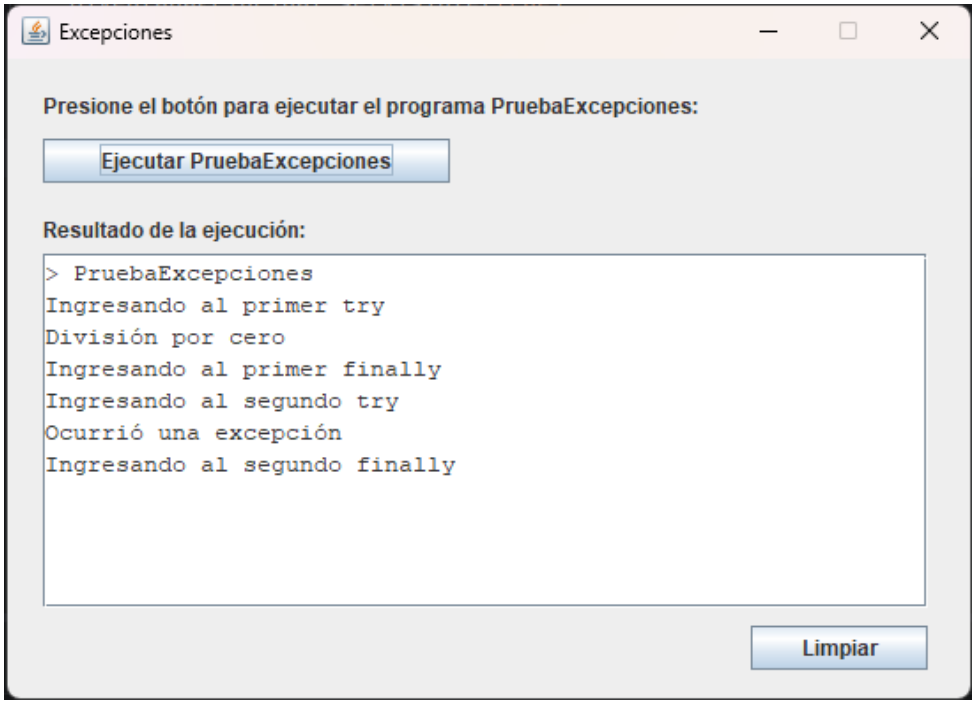


Figura 3: Ejecución de `PruebaExcepciones`: resultado del programa del libro.

2. Ejercicio 6.5: Lanzamiento de excepciones

Se define la clase `Vendedor` con nombre, apellidos y edad, un constructor que inicializa el nombre y los apellidos, el método `Imprimir` y el método `VerificarEdad`, que lanza una `IllegalArgumentException` si la edad no está entre 0 y 120 o si el vendedor no es mayor de 18 años. Los datos que en el libro se leen por teclado se ingresan en la ventana; si la edad no es válida se muestra la excepción lanzada. En el libro primero se revisa si la edad es menor de 18, de modo que una edad negativa muestra el mensaje equivocado; aquí primero se revisa el rango de 0 a 120.

2.1. Código fuente

Código 4: Vendedor.java (sin métodos get y set)

```
1  package Ejercicio6_5;
2
3  //Creación de la clase Vendedor.
4  public class Vendedor {
5
6      //Sección para declaración de atributos.
7      String Nombre;
8      String Apellidos;
9      int Edad;
10
11     //Creamos el constructor que inicializa el nombre y los apellidos
           del vendedor.
12     public Vendedor(String Nombre, String Apellidos){
13
14         //Asignamos los valores recibidos a los atributos.
15         this.Nombre = Nombre;
16         this.Apellidos = Apellidos;
17     }
18
19     //Métodos get y set omitidos en este documento (se encuentran en el
           código fuente del repositorio).
20
21     //Creamos un método que retorna los datos del vendedor para
           imprimirlos en la ventana.
22     public String Imprimir(){
23
24         //Inicializamos una variable con los datos del vendedor.
25         String Datos = "Nombre del vendedor = " + Nombre + "\n"
26                 + "Apellidos del vendedor = " + Apellidos + "\n"
27                 + "Edad del vendedor = " + Edad;
28
29         //Retornamos la variable.
30         return Datos;
31     }
32
33     //Creamos un método que verifica la edad del vendedor y lanza las
           excepciones correspondientes.
34     public void VerificarEdad(int Edad){
35
36         //La edad debe estar en el rango de 0 a 120, si no se cumple se
               lanza una excepción de argumento ilegal.
37         if (Edad < 0 || Edad > 120){
```

```
            throw new IllegalArgumentException("La edad no puede ser negativa ni mayor a 120.");
        }

        //El vendedor debe ser mayor de edad, si no se cumple se lanza una excepción de argumento ilegal.
        if (Edad < 18){
            throw new IllegalArgumentException("El vendedor debe ser mayor de 18 años.");
        }

        //Si la edad cumple los requisitos se asigna al vendedor.
        this.Edad = Edad;
    }
}
```

Código 5: VentanaPrincipal.java

```
package Ejercicio6_5;

//Importamos las clases necesarias para la interfaz gráfica y los eventos.
import java.awt.Container;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;

//Creación de la clase VentanaPrincipal (ventana para ingresar los datos de un vendedor y verificar su edad).
public class VentanaPrincipal extends JFrame implements ActionListener {

    //Sección para declaración de atributos.
    Container Contenedor; //Contenedor de los elementos gráficos.
    JLabel Nombre, Apellidos, Edad, TituloDatos; //Etiquetas de la ventana.
    JTextField CampoNombre, CampoApellidos, CampoEdad; //Campos de ingreso de los datos.
    JButton VerificarEdad; //Botón para crear el vendedor y verificar su edad.
    JTextArea Datos; //Área de texto donde se muestran los datos del vendedor.

    //Creamos el constructor de la ventana.
    public VentanaPrincipal(){

        //Creamos los componentes gráficos de la ventana.
        Inicio();

        //Establecemos el título, el tamaño y la posición de la ventana.
        setTitle("Vendedor");
        setSize(440, 330);
        setLocationRelativeTo(null);

```

```
35          //Establecemos que el botón de cerrar termina la aplicación y
                que el tamaño no se puede cambiar.
36          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
37          setResizable(false);
38      }
39
40      //Creamos un método para crear la ventana con sus componentes grá
            ficos.
41      private void Inicio(){
42
43          //Obtenemos el panel de contenidos de la ventana y le quitamos
                el layout.
44          Contenedor = getContentPane();
45          Contenedor.setLayout(null);
46
47          //Establecemos la etiqueta y el campo de texto para el nombre.
48          Nombre = new JLabel();
49          Nombre.setText("Nombre:");
50          Nombre.setBounds(20, 20, 120, 23);
51          CampoNombre = new JTextField();
52          CampoNombre.setBounds(130, 20, 270, 23);
53
54          //Establecemos la etiqueta y el campo de texto para los
                apellidos.
55          Apellidos = new JLabel();
56          Apellidos.setText("Apellidos:");
57          Apellidos.setBounds(20, 50, 120, 23);
58          CampoApellidos = new JTextField();
59          CampoApellidos.setBounds(130, 50, 270, 23);
60
61          //Establecemos la etiqueta y el campo de texto para la edad.
62          Edad = new JLabel();
63          Edad.setText("Edad:");
64          Edad.setBounds(20, 80, 120, 23);
65          CampoEdad = new JTextField();
66          CampoEdad.setBounds(130, 80, 270, 23);
67
68          //Establecemos el botón para verificar la edad y le agregamos el
                oyente de eventos.
69          VerificarEdad = new JButton();
70          VerificarEdad.setText("Verificar edad");
71          VerificarEdad.setBounds(130, 120, 180, 25);
72          VerificarEdad.addActionListener(this);
73
74          //Establecemos el área de texto donde se muestran los datos del
                vendedor.
75          TituloDatos = new JLabel();
76          TituloDatos.setText("Datos del vendedor:");
77          TituloDatos.setBounds(20, 160, 200, 23);
78          Datos = new JTextArea();
79          Datos.setEditable(false);
80          Datos.setBounds(20, 185, 380, 80);
81
82          //Añadimos cada componente gráfico al contenedor de la ventana.
83          Contenedor.add(Nombre);
84          Contenedor.add(CampoNombre);
85          Contenedor.add(Apellidos);
86          Contenedor.add(CampoApellidos);
```

```
87          Contenedor.add(Edad);
88          Contenedor.add(CampoEdad);
89          Contenedor.add(VerificarEdad);
90          Contenedor.add(TituloDatos);
91          Contenedor.add(Datos);
92      }
93
94      //Creamos un método para gestionar los eventos generados en la
            ventana.
95      @Override
96      public void actionPerformed(ActionEvent Evento){
97
98          //Si se pulsa Verificar edad se crea el vendedor y luego se
                verifica su edad.
99          if (Evento.getSource() == VerificarEdad){
100             try {
101                 //Obtenemos y convertimos la edad ingresada.
102                 int ValorEdad = Integer.parseInt(CampoEdad.getText().
                        trim());
103
104                 //Creamos el vendedor y verificamos su edad.
105                 Vendedor MiVendedor = new Vendedor(CampoNombre.getText()
                        , CampoApellidos.getText());
106                 MiVendedor.VerificarEdad(ValorEdad);
107
108                 //Si no se lanzó ninguna excepción mostramos los datos
                        del vendedor.
109                 Datos.setText(MiVendedor.Imprimir());
110             } catch (NumberFormatException Error){
111                 //Si la edad no es un número entero se muestra un
                        mensaje de error.
112                 JOptionPane.showMessageDialog(this, "La edad debe ser un
                         número entero.", "Error", JOptionPane.ERROR_MESSAGE)
                        ;
113             } catch (IllegalArgumentException Error){
114                 //Si la edad no cumple los requisitos se muestra la
                        excepción lanzada.
115                 Datos.setText("");
116                 JOptionPane.showMessageDialog(this, "
                        IllegalArgumentException: " + Error.getMessage(), "
                        Excepción", JOptionPane.ERROR_MESSAGE);
117             }
118         }
119     }
120 }
```

Código 6: Main.java

```
1  package Ejercicio6_5;
2
3  //Creación clase Main.
4  public class Main {
5      public static void main(String[] args){
6
7          //Creación de la ventana principal.
8          VentanaPrincipal MiVentanaPrincipal = new VentanaPrincipal();
9
10         //Hacemos visible la ventana principal.
```

```
11          MiVentanaPrincipal.setVisible(true);
12
13      }
14  }
```

2.2. Diagrama de clases

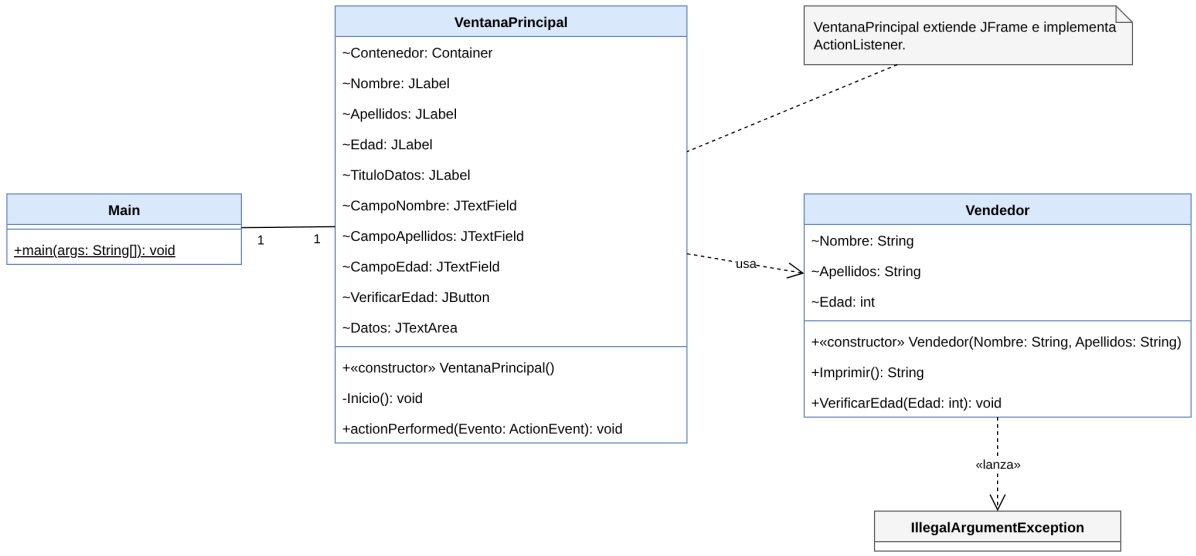


Figura 4: Diagrama de clases del ejercicio 6.5 (sin métodos get y set).

2.3. Diagrama de casos de uso

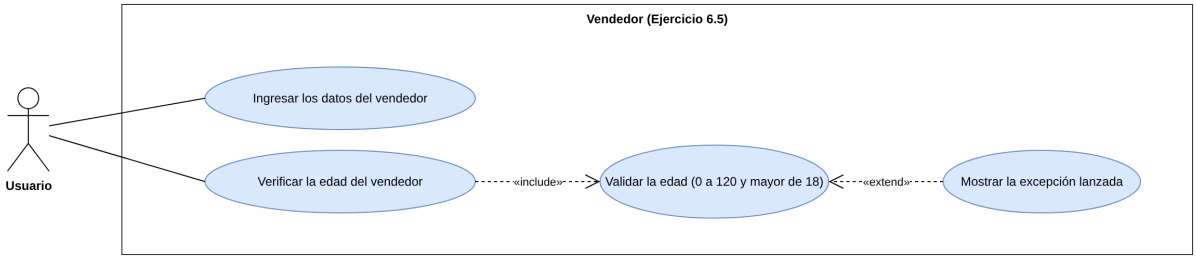


Figura 5: Diagrama de casos de uso del ejercicio 6.5.

2.4. Interfaz de usuario

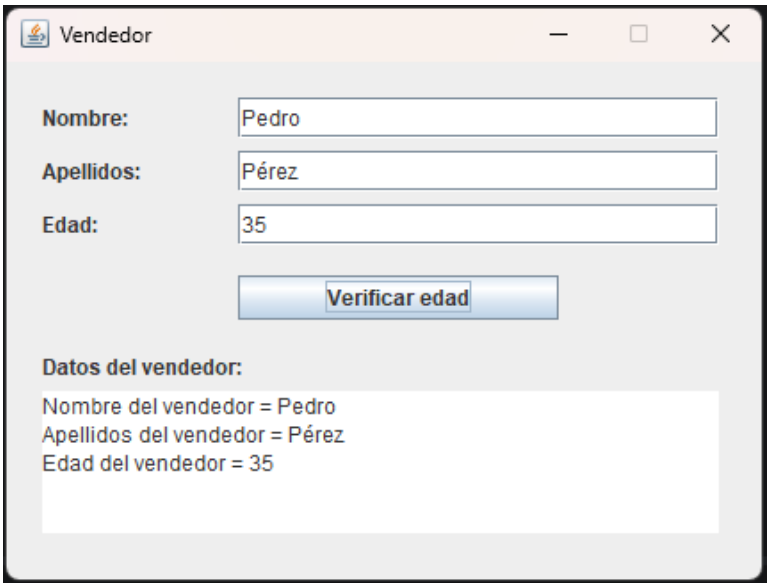


Figura 6: Vendedor con edad válida.

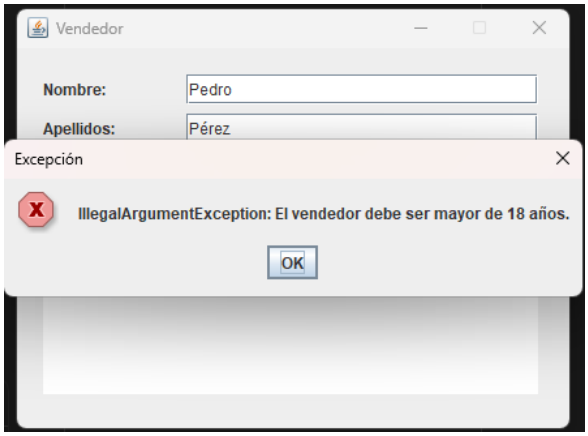


Figura 7: Excepción: el vendedor debe ser mayor de 18 años.

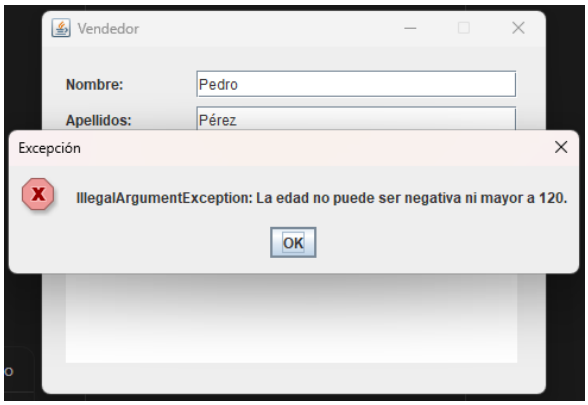


Figura 8: Excepción: la edad no puede ser negativa ni mayor a 120.

3. Ejercicio 6.6: Catchs múltiples

Se define la clase `CalculosNumericos` con dos métodos estáticos para calcular el logaritmo neperiano y la raíz cuadrada de un valor, que lanzan una `ArithmeticException` si el valor no es positivo (en el logaritmo también se rechaza el cero, porque su logaritmo no está definido). En la ventana se ingresa el valor numérico y cada cálculo tiene varios bloques *catch*: uno para la excepción aritmética y otro para el dato no numérico (`NumberFormatException`, el equivalente de `InputMismatchException` al leer un campo de texto). Los mensajes de error se muestran en rojo.

3.1. Código fuente

Código 7: CalculosNumericos.java

```
package Ejercicio6_6;

//Creación de la clase CalculosNumericos (cálculos matemáticos que lanzan excepciones si los datos no son correctos).
public class CalculosNumericos {

    //Creamos un método para calcular el logaritmo neperiano de un valor.
    public static double CalcularLogaritmoNeperiano(double Valor){

        //Si el valor no es positivo se lanza una excepción aritmética.
        if (Valor <= 0){
            throw new ArithmeticException("El valor debe ser un número positivo para calcular el logaritmo");
        }

        //Inicializamos una variable para guardar el cálculo del logaritmo neperiano.
        double Resultado = Math.log(Valor);

        //Retornamos la variable.
        return Resultado;
    }

    //Creamos un método para calcular la raíz cuadrada de un valor.
    public static double CalcularRaizCuadrada(double Valor){

        //Si el valor es negativo se lanza una excepción aritmética.
        if (Valor < 0){
            throw new ArithmeticException("El valor debe ser un número positivo para calcular la raíz cuadrada");
        }

        //Inicializamos una variable para guardar el cálculo de la raíz cuadrada.
        double Resultado = Math.sqrt(Valor);

        //Retornamos la variable.
        return Resultado;
    }
}
```

Código 8: VentanaPrincipal.java

```java
package Ejercicio6_6;

//Importamos las clases necesarias para la interfaz gráfica y los eventos.
import java.awt.Color;
import java.awt.Container;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;

//Creación de la clase VentanaPrincipal (ventana para calcular el logaritmo neperiano y la raíz cuadrada de un valor).
public class VentanaPrincipal extends JFrame implements ActionListener {

    //Sección para declaración de atributos.
    Container Contenedor; //Contenedor de los elementos gráficos.
    JLabel Valor; //Etiqueta del valor a ingresar.
    JLabel Logaritmo, Raiz; //Etiquetas de los resultados.
    JTextField CampoValor; //Campo de ingreso del valor numérico.
    JButton Calcular; //Botón para hacer los cálculos.

    //Creamos el constructor de la ventana.
    public VentanaPrincipal(){

        //Creamos los componentes gráficos de la ventana.
        Inicio();

        //Establecemos el título, el tamaño y la posición de la ventana.
        setTitle("Cálculos numéricos");
        setSize(520, 200);
        setLocationRelativeTo(null);

        //Establecemos que el botón de cerrar termina la aplicación y que el tamaño no se puede cambiar.
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
    }

    //Creamos un método para crear la ventana con sus componentes gráficos.
    private void Inicio(){

        //Obtenemos el panel de contenidos de la ventana y le quitamos el layout.
        Contenedor = getContentPane();
        Contenedor.setLayout(null);

        //Establecemos la etiqueta y el campo de texto para el valor numérico.
        Valor = new JLabel();
        Valor.setText("Valor numérico:");
        Valor.setBounds(20, 20, 140, 23);
        CampoValor = new JTextField();
        CampoValor.setBounds(160, 20, 150, 23);
```

```
52
53          //Establecemos el botón Calcular y le agregamos el oyente de
                eventos.
54          Calcular = new JButton();
55          Calcular.setText("Calcular");
56          Calcular.setBounds(330, 20, 150, 23);
57          Calcular.addActionListener(this);
58
59          //Establecemos las etiquetas de los resultados.
60          Logaritmo = new JLabel();
61          Logaritmo.setText("Logaritmo neperiano =");
62          Logaritmo.setBounds(20, 70, 470, 23);
63          Raiz = new JLabel();
64          Raiz.setText("Raíz cuadrada =");
65          Raiz.setBounds(20, 100, 470, 23);
66
67          //Añadimos cada componente gráfico al contenedor de la ventana.
68          Contenedor.add(Valor);
69          Contenedor.add(CampoValor);
70          Contenedor.add(Calcular);
71          Contenedor.add(Logaritmo);
72          Contenedor.add(Raiz);
73      }
74
75      //Creamos un método que muestra un resultado en una etiqueta (en
            rojo si es un error).
76      private void MostrarResultado(JLabel Etiqueta, String Texto, boolean
            EsError){
77
78          //Escribimos el texto y elegimos su color.
79          Etiqueta.setText(Texto);
80          if (EsError){
81              Etiqueta.setForeground(Color.RED);
82          } else {
83              Etiqueta.setForeground(Color.BLACK);
84          }
85      }
86
87      //Creamos un método para gestionar los eventos generados en la
            ventana.
88      @Override
89      public void actionPerformed(ActionEvent Evento){
90
91          //Si se pulsa Calcular se calculan el logaritmo neperiano y la
                raíz cuadrada, cada uno con sus catch.
92          if (Evento.getSource() == Calcular){
93
94              //Cálculo del logaritmo neperiano.
95              try {
96                  double ValorNumerico = Double.parseDouble(CampoValor.
                        getText().trim());
97                  double Resultado = CalculosNumericos.
                        CalcularLogaritmoNeperiano(ValorNumerico);
98                  MostrarResultado(Logaritmo, "Logaritmo neperiano = " +
                        String.format("%.4f", Resultado), false);
99              } catch (ArithmeticException Error){
100                 //Si el valor no es positivo se muestra el mensaje de la
                        excepción aritmética.
```

```
                MostrarResultado(Logaritmo, Error.getMessage(), true);
            } catch (NumberFormatException Error){
                //Si el valor no es numérico se muestra el mensaje correspondiente.
                MostrarResultado(Logaritmo, "El valor debe ser numérico para calcular el logaritmo", true);
            }

            //Cálculo de la raíz cuadrada.
            try {
                double ValorNumerico = Double.parseDouble(CampoValor.getText().trim());
                double Resultado = CalculosNumericos.CalcularRaizCuadrada(ValorNumerico);
                MostrarResultado(Raiz, "Raíz cuadrada = " + String.format("%.4f", Resultado), false);
            } catch (ArithmeticException Error){
                //Si el valor es negativo se muestra el mensaje de la excepción aritmética.
                MostrarResultado(Raiz, Error.getMessage(), true);
            } catch (NumberFormatException Error){
                //Si el valor no es numérico se muestra el mensaje correspondiente.
                MostrarResultado(Raiz, "El valor debe ser numérico para calcular la raíz cuadrada", true);
            }
        }
    }
}
```

Código 9: Main.java

```
package Ejercicio6_6;

//Creación clase Main.
public class Main {
    public static void main(String[] args){

        //Creación de la ventana principal.
        VentanaPrincipal MiVentanaPrincipal = new VentanaPrincipal();

        //Hacemos visible la ventana principal.
        MiVentanaPrincipal.setVisible(true);

    }
}
```

3.2. Diagrama de clases

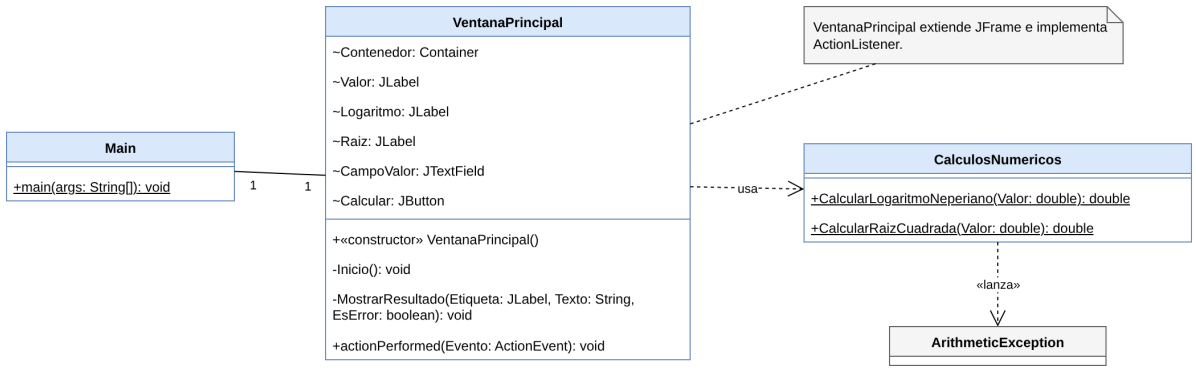


Figura 9: Diagrama de clases del ejercicio 6.6 (sin métodos get y set).

3.3. Diagrama de casos de uso

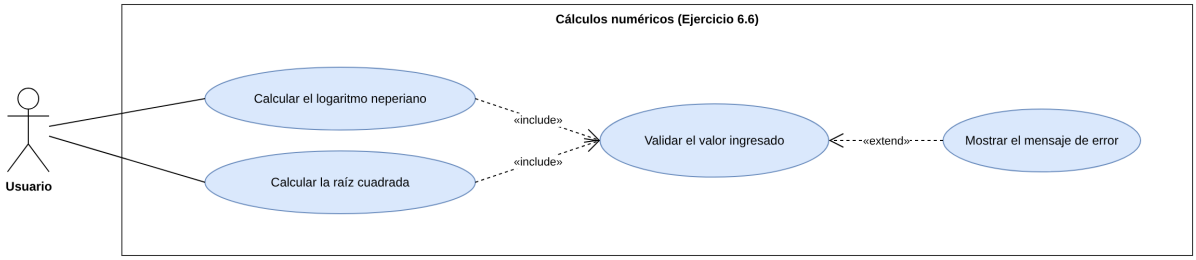


Figura 10: Diagrama de casos de uso del ejercicio 6.6.

3.4. Interfaz de usuario

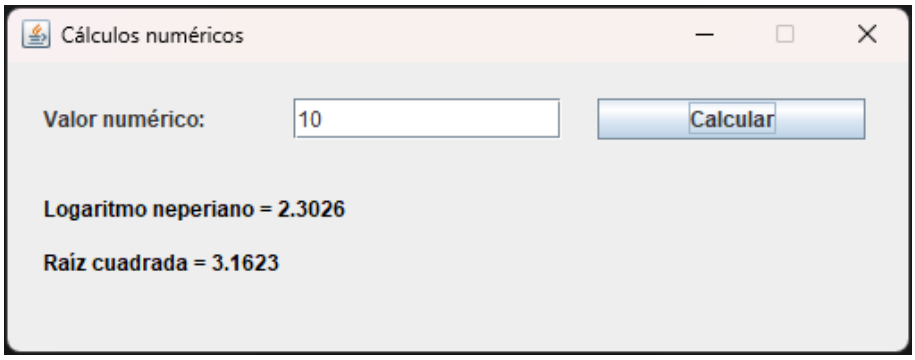


Figura 11: Logaritmo neperiano y raíz cuadrada de 10.

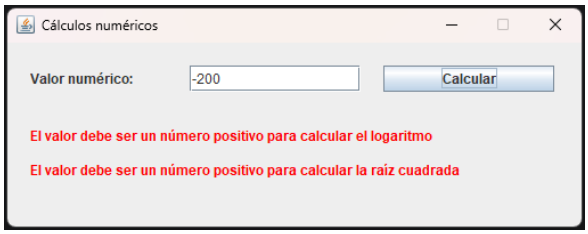


Figura 12: Excepciones aritméticas con el valor -200 (ejemplo del libro).

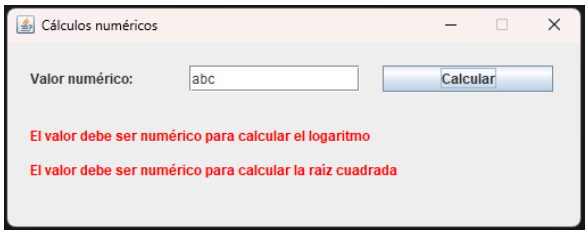


Figura 13: Valor no numérico: se ejecuta el otro *catch*.

4. Ejercicio 6.7: Validación de campos

Se definen las clases `EquipoMaratonProgramacion` y `Programador`. El equipo tiene nombre, universidad, lenguaje de programación y de dos a tres programadores, con métodos para saber si está lleno, para añadir programadores (excepción si está completo) y para validar el nombre y los apellidos de cada programador (excepción si tienen dígitos o si su longitud es igual o superior a 20 caracteres). Frente al libro se corrigen tres detalles: el constructor asignaba `this.universidad = this.universidad`, la longitud se validaba con `> 20` aunque el enunciado dice "igual o superior a 20", y no se verificaba el mínimo de dos programadores, que aquí se revisa al registrar el equipo. Además, los datos del equipo son campos obligatorios.

4.1. Código fuente

Código 10: EquipoMaratonProgramacion.java (sin métodos get y set)

```java
package Ejercicio6_7;

//Creación de la clase EquipoMaratonProgramacion (equipo de
    programadores que participa en una maratón de programación).
public class EquipoMaratonProgramacion {

    //Sección para declaración de atributos.
    String NombreEquipo;
    String Universidad;
    String LenguajeProgramacion;
    Programador[] Programadores; //Programadores que conforman el equipo
        .
    int TamanioEquipo; //Cantidad de programadores que hay en el equipo.

    //Creamos el constructor que inicializa los atributos del equipo.
    public EquipoMaratonProgramacion(String NombreEquipo, String
        Universidad, String LenguajeProgramacion){

        //Asignamos los valores recibidos a los atributos.
        this.NombreEquipo = NombreEquipo;
        this.Universidad = Universidad;
        this.LenguajeProgramacion = LenguajeProgramacion;

        //El tamaño del equipo inicialmente es cero y se crea un array
            para máximo tres programadores.
        TamanioEquipo = 0;
        Programadores = new Programador[3];
    }

    //Métodos get y set omitidos en este documento (se encuentran en el
        código fuente del repositorio).

    //Creamos un método para determinar si el equipo está lleno (tiene
        tres programadores).
    public boolean EstaLleno(){

        //Retornamos si el tamaño del equipo es igual a la capacidad del
            array.
        return TamanioEquipo == Programadores.length;
    }
```

```
34
35      //Creamos un método para añadir un programador al equipo.
36      public void Aniadir(Programador MiProgramador) throws Exception {
37
38          //Si el equipo está lleno se lanza la excepción correspondiente.
39          if (EstaLleno()){
40              throw new Exception("El equipo está completo. No se pudo
                    agregar programador.");
41          }
42
43          //Asignamos el programador al array y aumentamos el tamaño del
                equipo.
44          Programadores[TamanioEquipo] = MiProgramador;
45          TamanioEquipo = TamanioEquipo + 1;
46      }
47
48      //Creamos un método para verificar que el equipo tenga mínimo dos
            programadores.
49      public void VerificarTamanioMinimo() throws Exception {
50
51          //Si el equipo tiene menos de dos programadores se lanza la
                excepción correspondiente.
52          if (TamanioEquipo < 2){
53              throw new Exception("El equipo debe tener mínimo dos
                    programadores.");
54          }
55      }
56
57      //Creamos un método para validar que un campo obligatorio no esté
            vacío.
58      public static void ValidarObligatorio(String Campo, String
            NombreCampo) throws Exception {
59
60          //Si el campo está vacío se lanza la excepción correspondiente.
61          if (Campo.trim().isEmpty()){
62              throw new Exception("El campo " + NombreCampo + " es
                    obligatorio.");
63          }
64      }
65
66      //Creamos un método para validar el nombre o los apellidos de un
            programador.
67      public static void ValidarCampo(String Campo) throws Exception {
68
69          //El campo es obligatorio, si está vacío se lanza la excepción
                correspondiente.
70          if (Campo.trim().isEmpty()){
71              throw new Exception("El nombre y los apellidos no pueden
                    estar vacíos.");
72          }
73
74          //Recorremos el campo, si un carácter es un dígito se lanza la
                excepción correspondiente.
75          for (int j = 0; j < Campo.length(); j++){
76              char Caracter = Campo.charAt(j);
77              if (Character.isDigit(Caracter)){
78                  throw new Exception("El campo \"" + Campo + "\" no puede
                        tener dígitos.");
```

```
            }
        }

        //Si la longitud del campo es igual o superior a 20 caracteres se lanza la excepción correspondiente.
        if (Campo.length() >= 20){
            throw new Exception("La longitud del campo no debe ser igual o superior a 20 caracteres.");
        }
    }

    //Creamos un método que retorna los datos del equipo para imprimirlos en la ventana.
    public String Imprimir(){

        //Inicializamos una variable con los datos del equipo.
        String Datos = "Equipo: " + NombreEquipo + "\n"
                + "Universidad: " + Universidad + "\n"
                + "Lenguaje de programación: " + LenguajeProgramacion + "\n"
                + "Integrantes (" + TamanioEquipo + "):\n";

        //Recorremos los programadores agregando el nombre y los apellidos de cada uno.
        for (int i = 0; i < TamanioEquipo; i++){
            Datos = Datos + "   " + (i + 1) + ". " + Programadores[i].GetNombre() + " " + Programadores[i].GetApellidos() + "\n";
        }

        //Retornamos la variable.
        return Datos;
    }
}
```

Código 11: Programador.java (sin métodos get y set)

```
package Ejercicio6_7;

//Creación de la clase Programador (integrante de un equipo de la maratón de programación).
public class Programador {

    //Sección para declaración de atributos.
    String Nombre;
    String Apellidos;

    //Creamos el constructor que inicializa el nombre y los apellidos del programador.
    public Programador(String Nombre, String Apellidos){

        //Asignamos los valores recibidos a los atributos.
        this.Nombre = Nombre;
        this.Apellidos = Apellidos;
    }

    //Métodos get y set omitidos en este documento (se encuentran en el código fuente del repositorio).
```

```
19
20 }
```

Código 12: VentanaPrincipal.java

```
1  package Ejercicio6_7;
2
3  //Importamos las clases necesarias para la interfaz gráfica y los
       eventos.
4  import java.awt.Container;
5  import java.awt.event.ActionEvent;
6  import java.awt.event.ActionListener;
7  import javax.swing.JButton;
8  import javax.swing.JFrame;
9  import javax.swing.JLabel;
10 import javax.swing.JOptionPane;
11 import javax.swing.JScrollPane;
12 import javax.swing.JTextArea;
13 import javax.swing.JTextField;
14
15 //Creación de la clase VentanaPrincipal (ventana para crear un equipo de
       la maratón y añadir sus integrantes).
16 public class VentanaPrincipal extends JFrame implements ActionListener {
17
18     //Sección para declaración de atributos.
19     Container Contenedor; //Contenedor de los elementos gráficos.
20     JLabel DatosEquipo, NombreEquipo, Universidad, Lenguaje; //Etiquetas
           de los datos del equipo.
21     JLabel DatosIntegrante, Nombre, Apellidos, Estado; //Etiquetas de
           los datos de cada integrante y del estado.
22     JTextField CampoNombreEquipo, CampoUniversidad, CampoLenguaje; //
           Campos de ingreso de los datos del equipo.
23     JTextField CampoNombre, CampoApellidos; //Campos de ingreso de los
           datos de cada integrante.
24     JButton CrearEquipo, AniadirIntegrante, RegistrarEquipo; //Botones
           de la ventana.
25     JTextArea Integrantes; //Área de texto con los integrantes del
           equipo.
26     JScrollPane Desplazamiento; //Barras de desplazamiento del área de
           texto.
27     EquipoMaratonProgramacion MiEquipo; //Equipo que se crea en la
           ventana.
28
29     //Creamos el constructor de la ventana.
30     public VentanaPrincipal(){
31
32         //Creamos los componentes gráficos de la ventana.
33         Inicio();
34
35         //Establecemos el título, el tamaño y la posición de la ventana.
36         setTitle("Equipo maratón de programación");
37         setSize(545, 500);
38         setLocationRelativeTo(null);
39
40         //Establecemos que el botón de cerrar termina la aplicación y
               que el tamaño no se puede cambiar.
41         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
42         setResizable(false);
```

```
43      }
44
45      //Creamos un método para crear la ventana con sus componentes grá
            ficos.
46      private void Inicio(){
47
48          //Obtenemos el panel de contenidos de la ventana y le quitamos
                el layout.
49          Contenedor = getContentPane();
50          Contenedor.setLayout(null);
51
52          //Establecemos las etiquetas y los campos de texto de los datos
                del equipo.
53          DatosEquipo = new JLabel();
54          DatosEquipo.setText("Datos del equipo");
55          DatosEquipo.setBounds(20, 10, 300, 23);
56          NombreEquipo = new JLabel();
57          NombreEquipo.setText("Nombre del equipo:");
58          NombreEquipo.setBounds(20, 40, 170, 23);
59          CampoNombreEquipo = new JTextField();
60          CampoNombreEquipo.setBounds(200, 40, 310, 23);
61          Universidad = new JLabel();
62          Universidad.setText("Universidad:");
63          Universidad.setBounds(20, 70, 170, 23);
64          CampoUniversidad = new JTextField();
65          CampoUniversidad.setBounds(200, 70, 310, 23);
66          Lenguaje = new JLabel();
67          Lenguaje.setText("Lenguaje de programación:");
68          Lenguaje.setBounds(20, 100, 180, 23);
69          CampoLenguaje = new JTextField();
70          CampoLenguaje.setBounds(200, 100, 310, 23);
71
72          //Establecemos el botón Crear equipo y le agregamos el oyente de
                eventos.
73          CrearEquipo = new JButton();
74          CrearEquipo.setText("Crear equipo");
75          CrearEquipo.setBounds(200, 132, 150, 25);
76          CrearEquipo.addActionListener(this);
77
78          //Establecemos las etiquetas y los campos de texto de los datos
                de cada integrante.
79          DatosIntegrante = new JLabel();
80          DatosIntegrante.setText("Datos del integrante");
81          DatosIntegrante.setBounds(20, 170, 300, 23);
82          Nombre = new JLabel();
83          Nombre.setText("Nombre:");
84          Nombre.setBounds(20, 200, 170, 23);
85          CampoNombre = new JTextField();
86          CampoNombre.setBounds(200, 200, 310, 23);
87          Apellidos = new JLabel();
88          Apellidos.setText("Apellidos:");
89          Apellidos.setBounds(20, 230, 170, 23);
90          CampoApellidos = new JTextField();
91          CampoApellidos.setBounds(200, 230, 310, 23);
92
93          //Establecemos los botones para añadir integrantes y registrar
                el equipo.
94          AniadirIntegrante = new JButton();
```

```
95          AniadirIntegrante.setText("Añadir integrante");
96          AniadirIntegrante.setBounds(200, 262, 150, 25);
97          AniadirIntegrante.addActionListener(this);
98          RegistrarEquipo = new JButton();
99          RegistrarEquipo.setText("Registrar equipo");
100         RegistrarEquipo.setBounds(360, 262, 150, 25);
101         RegistrarEquipo.addActionListener(this);
102
103         //Establecemos el área de texto con los integrantes y la
                etiqueta del estado del equipo.
104         Integrantes = new JTextArea();
105         Integrantes.setEditable(false);
106         Desplazamiento = new JScrollPane(Integrantes);
107         Desplazamiento.setBounds(20, 300, 490, 115);
108         Estado = new JLabel();
109         Estado.setText("Primero cree el equipo.");
110         Estado.setBounds(20, 422, 490, 23);
111
112         //Añadimos cada componente gráfico al contenedor de la ventana.
113         Contenedor.add(DatosEquipo);
114         Contenedor.add(NombreEquipo);
115         Contenedor.add(CampoNombreEquipo);
116         Contenedor.add(Universidad);
117         Contenedor.add(CampoUniversidad);
118         Contenedor.add(Lenguaje);
119         Contenedor.add(CampoLenguaje);
120         Contenedor.add(CrearEquipo);
121         Contenedor.add(DatosIntegrante);
122         Contenedor.add(Nombre);
123         Contenedor.add(CampoNombre);
124         Contenedor.add(Apellidos);
125         Contenedor.add(CampoApellidos);
126         Contenedor.add(AniadirIntegrante);
127         Contenedor.add(RegistrarEquipo);
128         Contenedor.add(Desplazamiento);
129         Contenedor.add(Estado);
130     }
131
132     //Creamos un método para mostrar cuántos integrantes tiene el equipo
            y si está lleno.
133     private void ActualizarEstado(){
134
135         //Escribimos el estado del equipo en la etiqueta.
136         Estado.setText("Integrantes: " + MiEquipo.GetTamanioEquipo() + "
                de 3   |   Equipo lleno: " + (MiEquipo.EstaLleno() ? "sí" :
               "no"));
137     }
138
139     //Creamos un método para mostrar una excepción en un cuadro de diá
            logo.
140     private void MostrarExcepcion(Exception Error){
141
142         //Mostramos el mensaje de la excepción.
143         JOptionPane.showMessageDialog(this, "Exception: " + Error.
               getMessage(), "Excepción", JOptionPane.ERROR_MESSAGE);
144     }
145
146     //Creamos un método para gestionar los eventos generados en la
```

```
        ventana.
147     @Override
148     public void actionPerformed(ActionEvent Evento){
149
150         //Si se pulsa Crear equipo se validan los campos obligatorios y
                se crea el equipo.
151         if (Evento.getSource() == CrearEquipo){
152             try {
153                 EquipoMaratonProgramacion.ValidarObligatorio(
                        CampoNombreEquipo.getText(), "nombre del equipo");
154                 EquipoMaratonProgramacion.ValidarObligatorio(
                        CampoUniversidad.getText(), "universidad");
155                 EquipoMaratonProgramacion.ValidarObligatorio(
                        CampoLenguaje.getText(), "lenguaje de programación");
156                 MiEquipo = new EquipoMaratonProgramacion(
                        CampoNombreEquipo.getText().trim(), CampoUniversidad.
                        getText().trim(), CampoLenguaje.getText().trim());
157                 Integrantes.setText("Equipo " + MiEquipo.GetNombreEquipo
                        () + " creado. Integrantes:\n");
158                 ActualizarEstado();
159             } catch (Exception Error){
160                 MostrarExcepcion(Error);
161             }
162         }
163
164         //Si se pulsa Añadir integrante se validan el nombre y los
                apellidos y se añade el programador.
165         if (Evento.getSource() == AniadirIntegrante){
166
167             //Si todavía no hay equipo no se puede añadir el integrante.
168             if (MiEquipo == null){
169                 JOptionPane.showMessageDialog(this, "Primero debe crear
                        el equipo.", "Error", JOptionPane.ERROR_MESSAGE);
170                 return;
171             }
172
173             try {
174                 String NombreIntegrante = CampoNombre.getText().trim();
175                 String ApellidosIntegrante = CampoApellidos.getText().
                        trim();
176                 EquipoMaratonProgramacion.ValidarCampo(NombreIntegrante)
                        ;
177                 EquipoMaratonProgramacion.ValidarCampo(
                        ApellidosIntegrante);
178                 MiEquipo.Aniadir(new Programador(NombreIntegrante,
                        ApellidosIntegrante));
179                 Integrantes.append("   " + MiEquipo.GetTamanioEquipo() +
                         ". " + NombreIntegrante + " " + ApellidosIntegrante
                        + "\n");
180                 CampoNombre.setText("");
181                 CampoApellidos.setText("");
182                 ActualizarEstado();
183             } catch (Exception Error){
184                 MostrarExcepcion(Error);
185             }
186         }
187
188         //Si se pulsa Registrar equipo se verifica que tenga mínimo dos
```

```
            integrantes y se muestran sus datos.
        if (Evento.getSource() == RegistrarEquipo){

            //Si todavía no hay equipo no se puede registrar.
            if (MiEquipo == null){
                JOptionPane.showMessageDialog(this, "Primero debe crear
                    el equipo.", "Error", JOptionPane.ERROR_MESSAGE);
                return;
            }

            try {
                MiEquipo.VerificarTamanioMinimo();
                JOptionPane.showMessageDialog(this, "Equipo registrado
                    correctamente.\n\n" + MiEquipo.Imprimir(), "Equipo
                    registrado", JOptionPane.INFORMATION_MESSAGE);
            } catch (Exception Error){
                MostrarExcepcion(Error);
            }
        }
    }
}
```

Código 13: Main.java

```
package Ejercicio6_7;

//Creación clase Main.
public class Main {
    public static void main(String[] args){

        //Creación de la ventana principal.
        VentanaPrincipal MiVentanaPrincipal = new VentanaPrincipal();

        //Hacemos visible la ventana principal.
        MiVentanaPrincipal.setVisible(true);

    }
}
```

4.2. Diagrama de clases

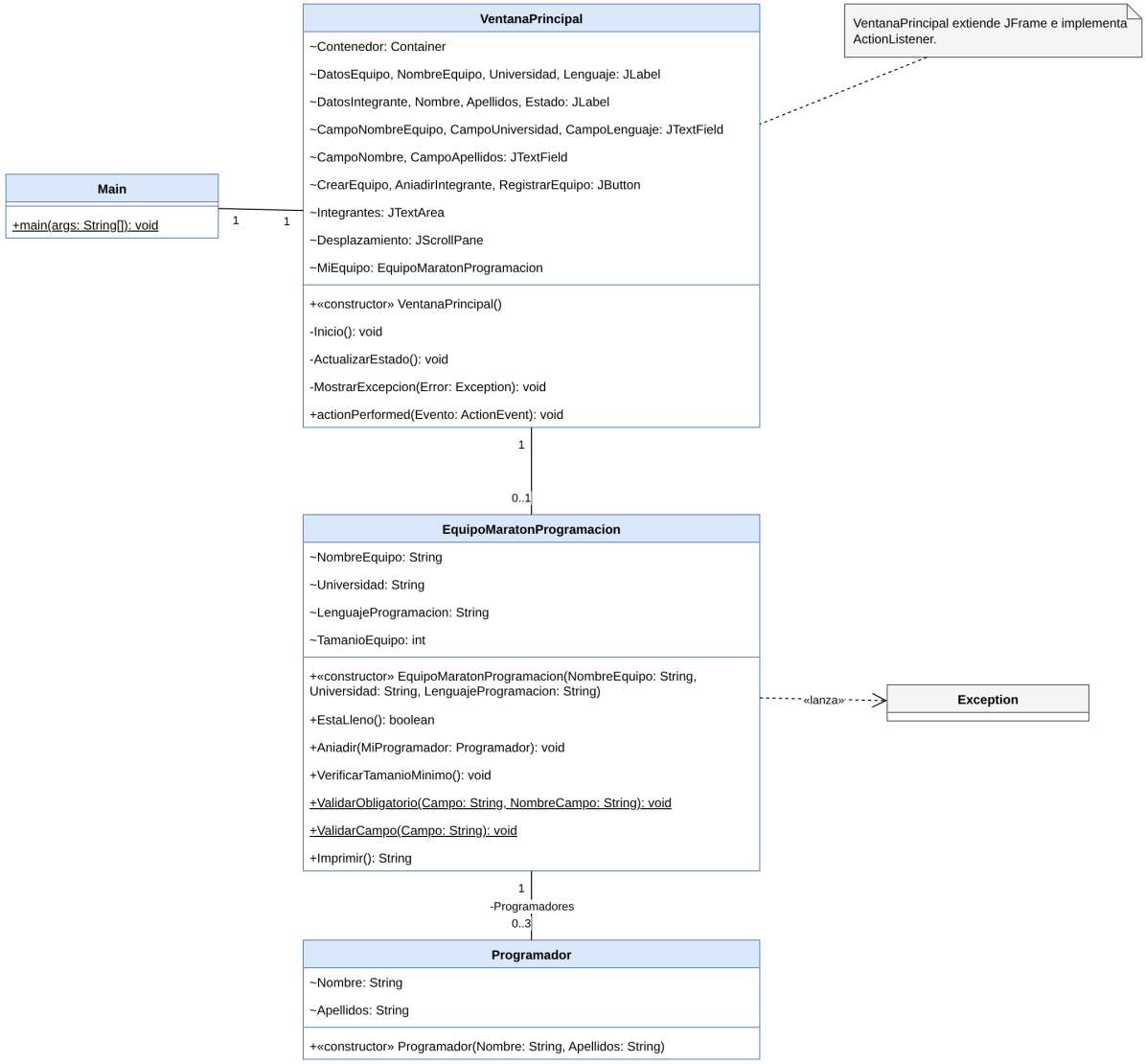


Figura 14: Diagrama de clases del ejercicio 6.7 (sin métodos get y set).

4.3. Diagrama de casos de uso

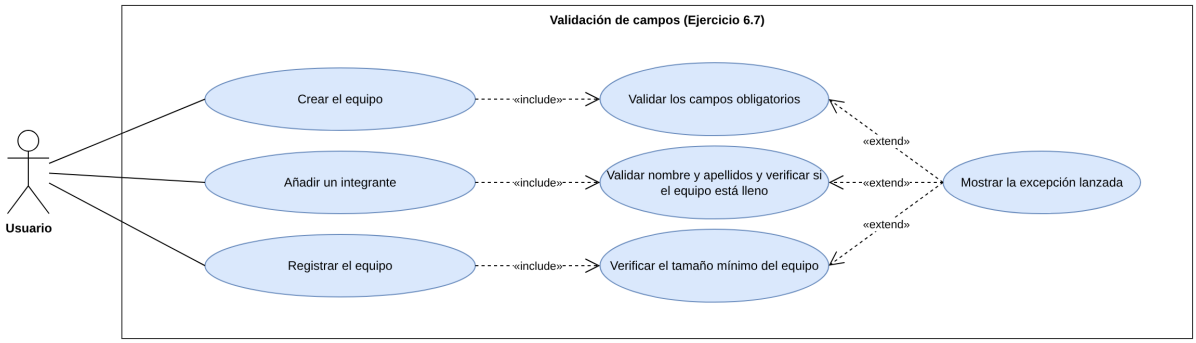


Figura 15: Diagrama de casos de uso del ejercicio 6.7.

4.4. Interfaz de usuario

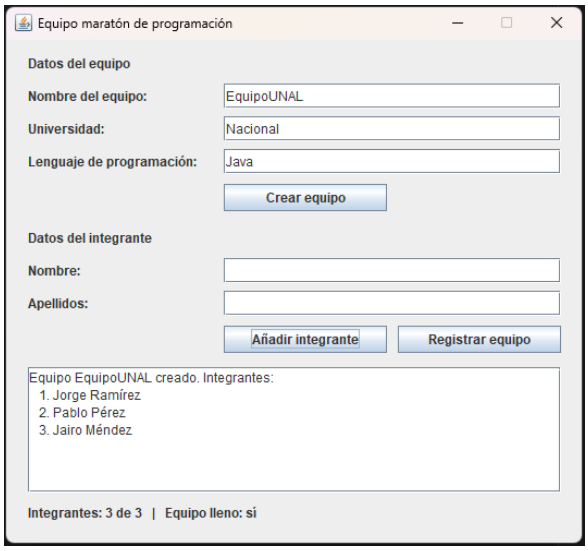


Figura 16: Equipo creado con sus tres integrantes.

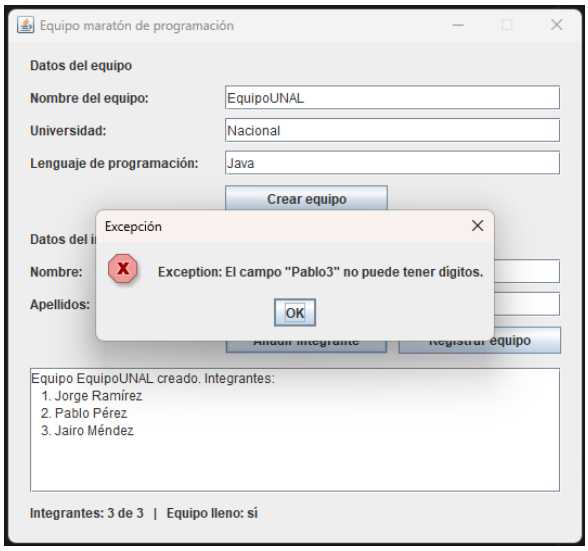


Figura 17: Excepción: el nombre de un integrante tiene dígitos.

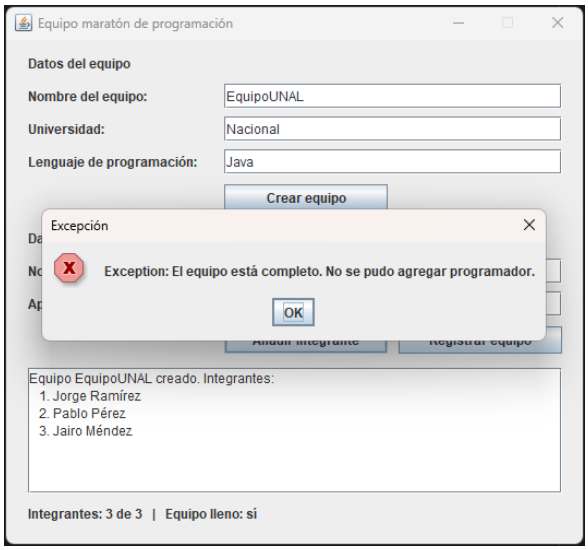


Figura 18: Excepción: el equipo está completo.

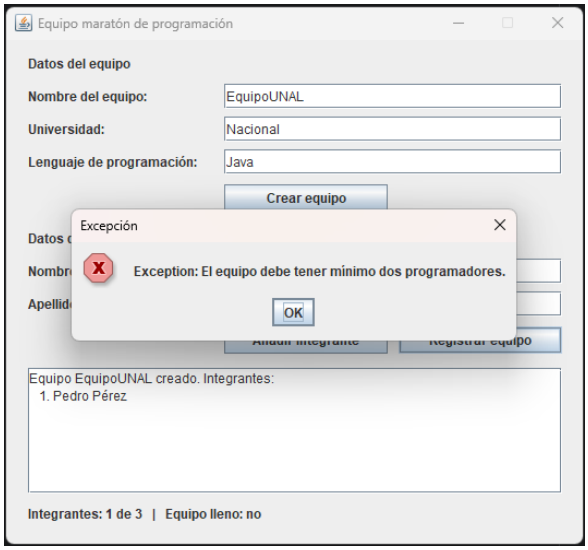


Figura 19: Excepción: el equipo debe tener mínimo dos programadores.

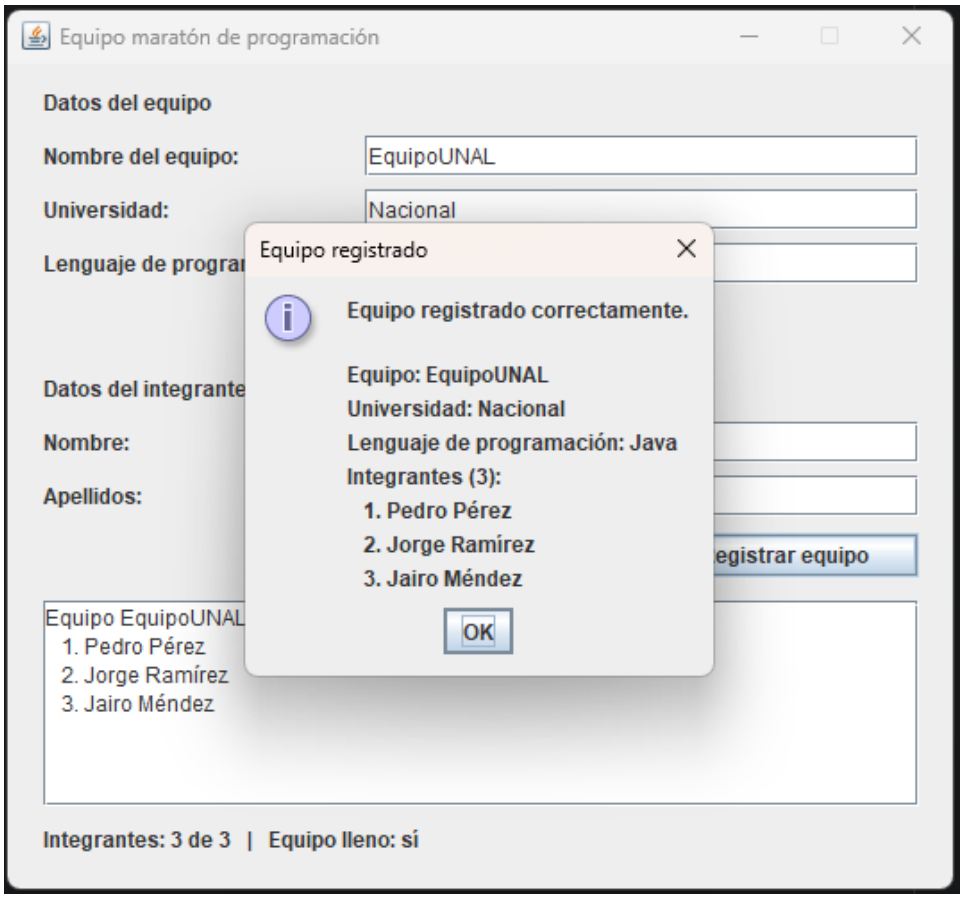


Figura 20: Registro del equipo con sus datos.

5. Ejercicio 6.8: Lectura de archivos

Se define la clase `LeerArchivo`, que lee el archivo de texto `prueba.txt` línea por línea con un flujo de bytes: `FileInputStream` obtiene los bytes del archivo, `InputStreamReader` los convierte en caracteres y `BufferedReader` los lee con `readLine()`. El contenido se muestra en la ventana al pulsar el botón Leer archivo. El archivo `prueba.txt` acompaña al programa (en el libro está en `C:/prueba.txt`); si no se puede leer se captura la `IOException` y se muestra el mensaje "No se pudo leer el archivo.".

5.1. Código fuente

Código 14: LeerArchivo.java

```
package Ejercicio6_8;

//Importamos las clases necesarias para leer archivos.
import java.io.BufferedReader;
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStreamReader;

//Creación de la clase LeerArchivo (lee los datos de un archivo de texto con un flujo de bytes).
public class LeerArchivo {

    //Creamos un método que lee un archivo de texto línea por línea y retorna su contenido.
    public static String Leer(String NombreArchivo) throws IOException {

        //Creamos el flujo de bytes, el conversor de bytes a caracteres y el búfer de lectura.
        FileInputStream Archivo = new FileInputStream(NombreArchivo);
        InputStreamReader Conversor = new InputStreamReader(Archivo, "UTF-8");
        BufferedReader Filtro = new BufferedReader(Conversor);

        //Inicializamos una variable para guardar el contenido del archivo.
        String Contenido = "";

        //Leemos la primera línea y, mientras haya líneas, las agregamos al contenido.
        String Linea = Filtro.readLine();
        while (Linea != null){
            Contenido = Contenido + Linea + "\n";
            Linea = Filtro.readLine();
        }

        //Cerramos el archivo.
        Filtro.close();

        //Retornamos el contenido del archivo.
        return Contenido;
    }
}
```

Código 15: VentanaPrincipal.java

```java
package Ejercicio6_8;

//Importamos las clases necesarias para la interfaz gráfica, los eventos y los archivos.
import java.awt.Container;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.File;
import java.io.IOException;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;

//Creación de la clase VentanaPrincipal (ventana para leer el archivo prueba.txt y mostrar su contenido).
public class VentanaPrincipal extends JFrame implements ActionListener {

    //Sección para declaración de atributos.
    Container Contenedor; //Contenedor de los elementos gráficos.
    JLabel Archivo, TituloContenido; //Etiquetas de la ventana.
    JButton Leer, Limpiar; //Botones para leer el archivo y para borrar el contenido mostrado.
    JTextArea Contenido; //Área de texto donde se muestra el contenido del archivo.
    JScrollPane Desplazamiento; //Barras de desplazamiento del área de texto.
    String NombreArchivo; //Ruta del archivo prueba.txt que se va a leer.

    //Creamos el constructor de la ventana.
    public VentanaPrincipal(){

        //Buscamos el archivo prueba.txt que acompaña al programa; si no se encuentra se deja solo el nombre.
        try {
            NombreArchivo = new File(getClass().getResource("prueba.txt").toURI()).getPath();
        } catch (Exception Error){
            NombreArchivo = "prueba.txt";
        }

        //Creamos los componentes gráficos de la ventana.
        Inicio();

        //Establecemos el título, el tamaño y la posición de la ventana.
        setTitle("Lectura de archivos");
        setSize(560, 400);
        setLocationRelativeTo(null);

        //Establecemos que el botón de cerrar termina la aplicación y que el tamaño no se puede cambiar.
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setResizable(false);
```

```
49      }
50
51      //Creamos un método para crear la ventana con sus componentes grá
            ficos.
52      private void Inicio(){
53
54          //Obtenemos el panel de contenidos de la ventana y le quitamos
                el layout.
55          Contenedor = getContentPane();
56          Contenedor.setLayout(null);
57
58          //Establecemos la etiqueta con el nombre del archivo que se va a
                leer.
59          Archivo = new JLabel();
60          Archivo.setText("Archivo: " + new File(NombreArchivo).getName())
                ;
61          Archivo.setToolTipText(NombreArchivo);
62          Archivo.setBounds(20, 20, 300, 23);
63
64          //Establecemos el botón Leer archivo y le agregamos el oyente de
                eventos.
65          Leer = new JButton();
66          Leer.setText("Leer archivo");
67          Leer.setBounds(320, 20, 200, 25);
68          Leer.addActionListener(this);
69
70          //Establecemos el área de texto donde se muestra el contenido
                del archivo.
71          TituloContenido = new JLabel();
72          TituloContenido.setText("Contenido del archivo:");
73          TituloContenido.setBounds(20, 60, 300, 23);
74          Contenido = new JTextArea();
75          Contenido.setEditable(false);
76          Contenido.setFont(new Font("Monospaced", Font.PLAIN, 13));
77          Desplazamiento = new JScrollPane(Contenido);
78          Desplazamiento.setBounds(20, 85, 500, 225);
79
80          //Establecemos el botón Limpiar y le agregamos el oyente de
                eventos.
81          Limpiar = new JButton();
82          Limpiar.setText("Limpiar");
83          Limpiar.setBounds(420, 320, 100, 25);
84          Limpiar.addActionListener(this);
85
86          //Añadimos cada componente gráfico al contenedor de la ventana.
87          Contenedor.add(Archivo);
88          Contenedor.add(Leer);
89          Contenedor.add(TituloContenido);
90          Contenedor.add(Desplazamiento);
91          Contenedor.add(Limpiar);
92      }
93
94      //Creamos un método para gestionar los eventos generados en la
            ventana.
95      @Override
96      public void actionPerformed(ActionEvent Evento){
97
98          //Si se pulsa Leer archivo se lee el archivo prueba.txt y se
```

```
            muestra su contenido.
99          if (Evento.getSource() == Leer){
100              try {
101                  Contenido.setText(LeerArchivo.Leer(NombreArchivo));
102              } catch (IOException Error){
103                  //Si el archivo no se puede leer se muestra un mensaje
                        de error.
104                  Contenido.setText("");
105                  JOptionPane.showMessageDialog(this, "No se pudo leer el
                        archivo.", "Error", JOptionPane.ERROR_MESSAGE);
106              }
107          }
108
109          //Si se pulsa Limpiar se borra el contenido mostrado.
110          if (Evento.getSource() == Limpiar){
111              Contenido.setText("");
112          }
113      }
114  }
```

Código 16: Main.java

```
1   package Ejercicio6_8;
2
3   //Creación clase Main.
4   public class Main {
5       public static void main(String[] args){
6
7           //Creación de la ventana principal.
8           VentanaPrincipal MiVentanaPrincipal = new VentanaPrincipal();
9
10          //Hacemos visible la ventana principal.
11          MiVentanaPrincipal.setVisible(true);
12
13      }
14  }
```

Código 17: prueba.txt (archivo de texto que lee el programa)

```
1   Línea 1
2   Línea 2
3   Línea 3
4   Línea 4
5   Línea 5
6   Línea 6
7   Línea 7
8   Línea 8
9   Línea 9
10  Línea 10
```

5.2. Diagrama de clases

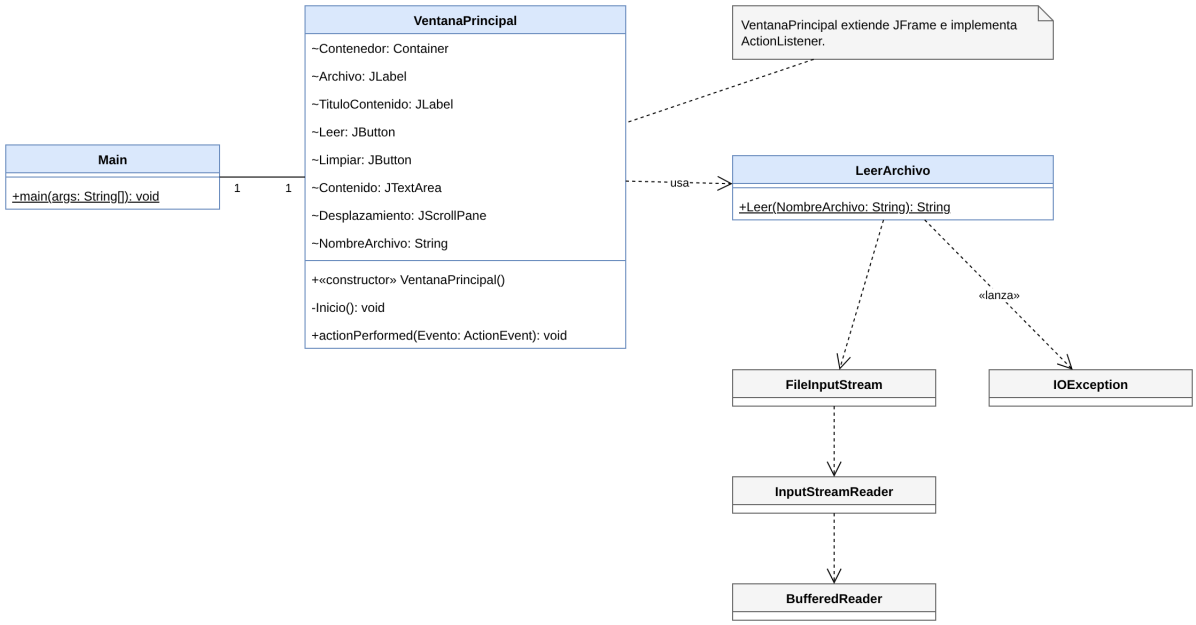


Figura 21: Diagrama de clases del ejercicio 6.8 (sin métodos get y set).

5.3. Diagrama de casos de uso

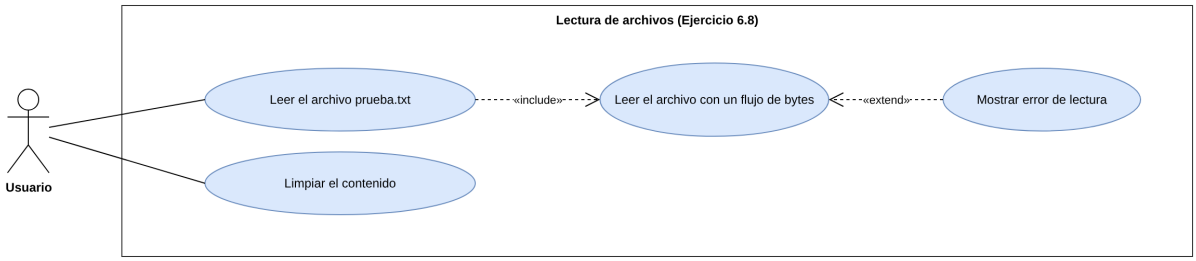


Figura 22: Diagrama de casos de uso del ejercicio 6.8.

5.4. Interfaz de usuario

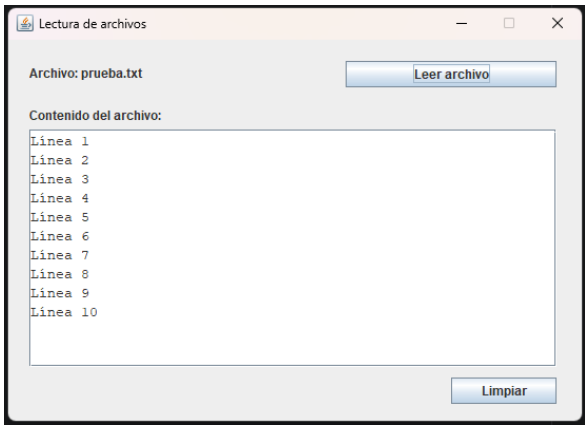


Figura 23: Lectura del archivo `prueba.txt`.



Figura 24: Mensaje de error cuando no se encuentra el archivo `prueba.txt`.